

程序设计实践报告

基于领域特定脚本语言的客服机器人的设计与实现

班级：2022211123

姓名：冯国晟

学号：2022211123

1 概述

1.1 作业描述

领域特定语言（Domain Specific Language, DSL）可以提供一种相对简单的文法，用于特定领域的业务流程定制。本作业要求定义一个领域特定脚本语言，这个语言能够描述在线客服机器人（机器人客服是目前提升客服效率的重要技术，在银行、通信和商务等领域的复杂信息系统中有着广泛的应用）的自动应答逻辑，并设计实现一个解释器解释执行这个脚本，可以根据用户的不同输入，根据脚本的逻辑设计给出相应的应答。

1.2 基本要求

脚本语言的语法可以自由定义，只要语义上满足描述客服机器人自动应答逻辑的要求。

程序输入输出形式不限，可以简化为纯命令行界面。

应该给出几种不同的脚本范例，对不同脚本范例解释器执行之后会有不同的行为表现。

1.3 实现环境

- Windows 11 v22631.4460
- Visual Studio Code v1.96.0
- gcc version 13.2.0

2 设计说明

2.1 脚本语言设计说明

考虑到存在给出不同的脚本范例，对不同的脚本范例解释器执行之后有不同的行为表现这一需求，自行设计如下脚本语言：对大小写敏感对换行敏感对缩进不敏感，可通过使用规范设计对应机器人应答模式与行为方式。

存在 STATE、CASE、DEFAULT、OUT、NEXT、BEGIN、EXIT 七个保留关键字，具体用法如下：

- STATE**

使用方法：

STATE 状态名

使用限制：

状态名 可自由选取任意非保留关键字且不包含空格字符串设置，默认脚本语言中第一个出现的 STATE 为机器人起始状态；

功能：

设置机器人对应状态行为规范;

- **CASE**

使用方法:

```
CASE "待匹配信息"  
    动作1  
    动作2  
    ...  
    动作n
```

使用限制:

`待匹配信息` 需由英文双引号 "" 括起, `待匹配信息` 可设置为任意字符串, 必须处在某一状态当中使用;

其下动作数量可任意设置;

功能:

当用户输入信息与 `待匹配信息` 匹配上时, 执行其下所有行动;

- **DEFAULT**

使用方法:

```
DEFAULT  
    动作1  
    动作2  
    ...  
    动作n
```

使用限制:

必须处在某一状态当中使用;

其下动作数量可任意设置;

功能:

当用户输入信息与所有 `CASE` 待匹配信息失配后执行其下所有动作;

- **OUT**

使用方法:

```
OUT "输出信息";
```

使用限制:

`输出信息` 需由英文双引号 "" 括起, `输出信息` 可设置为任意字符串, 默认不同 `OUT` 语句输出一行信息输出结束自动换行, 必须处在某一状态当中使用;

功能:

设置机器人输出信息;

- **NEXT**

使用方法:

```
NEXT 下一状态名;
```

使用限制:

`下一状态名` 可自由选取任意以声明或未声明的状态, 但若整个语法文件均为未声明该状态会致使自动机状态转移时抛出异常, `下一状态名` 需满足不含空格、不与保留关键字重复, 同时必须处在某一状态当中使用;

功能：

设置机器人执行完当前动作后的下一状态；

- **BEGIN**

使用方法：

现式的使用：

```
STATE "状态"
  CASE "BEGIN"
    动作1
    动作2
    ...
    动作n
```

隐式使用：

```
STATE "状态"
  动作1
  动作2
  ...
  动作n
```

使用限制：

上述两种使用方式效果等效，其下动作数量可任意限制，必须使用在某一状态当中；

功能：

设置当机器人到达这一状态时首先需执行的动作；

- **EXIT**

使用方法：

```
NEXT EXIT
```

使用限制：

必须在某一状态的某一动作当中使用，表明该动作为退出自动机

功能：

表明本次动作为退出自动机。

使用范例：

```
STATE s1
  OUT "abcd"
  CASE "text1"
    OUT "1"
    NEXT s1
  DEFAULT
    OUT "2"
    OUT "3"
    NEXT s2
STATE s2
  OUT "qwert"
  NEXT EXIT
```

上述脚本语言所定义的自动机行为如下：

起始状态为 `s1`

进入状态 `s1` 后首先输出内容 `abcd`，此时若检测到用户输入内容为 `text1` 则输出内容 `1` 转移到状态 `s1`，若检测到用户输入的内容为其他内容，则输出信息 `1` (换行) `2` 然后转移到状态 `s2`

进入状态 `s2` 后首先输出信息 `qwert`，接着退出自动机运行。

2.2 程序设计说明

2.2.1 风格说明

变量与函数命名上除临时变量 `i`、`j` 外函数名与变量名均采用大驼峰命名法，如 `SyntaxFileName` 对应语法文件名、`ShowSyntaxTree` 对应展示语法树信息等；

注释则采用 `Doxygen` 注释，以方便阅读理解对应文件、函数的功能、参数和返回值，并方便维护与扩展；

例如 `Syntax.cpp` 文件注释如下：

```
/**
 * @file Syntax.cpp
 * @brief 语法解析器实现，解析并构建状态转移树
 *
 * 该文件实现了 `SyntaxParser`
 * 主要用于构建语法树
 * 主要功能：
 *   - 构建语法树
 */
```

2.2.2 模块划分说明

整体上将程序计划分为 `Syntax`、`Automaton` 与 `DSL` 三个模块，其中：

- `Syntax` 对应实现根据输入的语法文件，解析并构建其对应的语法树功能；
- `Automaton` 对应实现语法树自动机处理，如根据输入处理状态转移等功能；
- `DSL` 对应具体实现客服机器人功能。

其中文件说明如下：

```
Syntax.h // 头文件
Syntax.cpp // Syntax模块的实现
Automaton.cpp // Automaton模块的实现
DSL.cpp // DSL模块的实现
```

2.2.3 数据结构说明

1. 关键数据结构概述

1. Action:

记录该动作对应的输出信息与下一状态

```
// 定义该状态下的动作信息
class Action {
public:
    std::vector<std::string> OutMessage; // 输出信息
    std::string NextState; // 下一状态
};
```

2. State:

记录该状态的动作表，用输入匹配内容映射对应的动作Action

```
// State 类表示一个状态，包含一张动作表
class State {
public:
    std::unordered_map<std::string, Action> ActionTable; // 对应输入
映射 对应动作
};
```

3. SyntaxParser:

语法解析器，用于解析语法文件并构建状态表，包含的内容详见注释

```
class SyntaxParser {
private:
    std::string SyntaxFileName; // 语法文件名
    std::ifstream SyntaxFile; // 语法文件文件流
    std::string BeginState; // 起始状态信息
    std::unordered_map<std::string, State> StateTable; // 状态表， 用状
态名字 映射 状态
    bool Begin; // 是否已经处理起始状态
    bool IsSyntaxTreeReady; // 判断语法树是否构建完成
};
```

4. Automaton:

表示基于语法树工作的自动机

```
class Automaton {
private:
    SyntaxParser Rules; // 语法解析器对象
    State NowState; // 当前状态
    bool Ready; // 自动机准备状态
};
```

2. 全局常量

1. 特殊符号:

提供统一的特殊符号定义，便于状态机识别并处理：

```
// 特殊符号定义，用于语法解析的特殊符号
const std::string BEGIN_SYMBOL = "BEGIN"; // 起始符号
const std::string EXIT_SYMBOL = "EXIT"; // 退出符号
const std::string DEFAULT_SYMBOL = "DEFAULT"; // 默认符号
```

2. 保留字映射:

将保留字与数字标号对应，方便语法解析器进行高效处理:

```
// 定义保留字及其对应的编号，便于语法解析器进行语法分析
const std::unordered_map<std::string, int> Remainword {
    {"STATE", 0}, {"OUT", 1}, {"NEXT", 2}, {"CASE", 3}, {"DEFAULT", 4},
    {"BEGIN", 5}, {"EXIT", 6}
};
```

2.2.4 API说明

1. SyntaxParser 类 API

1. 构造函数

```
SyntaxParser(const std::string& FileName);
```

功能：初始化语法解析器并准备语法文件构造语法树

参数：

FileName:语法文件文件名

2. CheckReady()

```
bool CheckReady(void);
```

功能：检查语法树是否构建正确完成

返回值：若语法树正确构建完成返回 `true`，若语法树构建错误返回 `false`

3. ShowSyntaxTree()

```
void ShowSyntaxTree(void);
```

功能：展示所构建的语法树信息

4. AskBegin()

```
std::string AskBegin(void) const
```

功能：查询语法树起始状态

返回值：该语法树的起始状态对应的字符串即起始状态名

5. AskState()

```
std::pair<bool, State> SyntaxParser::AskState(const std::string& ask);
```

功能：根据状态名查询状态

参数：

`ask`：查询状态的状态名

返回值：

为一个 `pair<bool, State>`，其 `first` 部表示查询状态，若查询成功则为 `true`，查询失败为 `false`

其 `second` 部为对应状态 `State`

2. Automaton 类 API

1. 构造函数

```
Automaton(const std::string& FileName, std::vector<std::string>& Buffer);
```

功能：基于语法文件初始化自动机与输出缓冲区

参数：

`FileName`：语法文件文件名

`Buffer`：输出缓冲区

2. CheckReady()

```
bool CheckReady(void) const;
```

功能：检查自动机是否准备好运行

返回值：若自动机已做好运行准备则返回 `true`，反之若未准备完毕则返回 `false`

3. Exit()

```
void Exit(void);
```

功能：退出自动机

4. Input()

```
void Input(const std::string& input, std::vector<std::string>& Buffer);
```

功能：根据输入的字符串 `input` 和当前状态进行状态转移

参数：

`input`：输入的字符串

`Buffer`：输出信息缓冲区

5. CalWordDis()

```
int CalWordDis(const std::string& temp, const std::string& check)
```

功能：计算 `temp` 与 `check` 两个字符串之间的最小编辑距离，用以支持模糊搜索匹配，处理含噪音的输入

参数：

`temp`：待匹配的第一个字符

`check`：待匹配的第二个字符

返回值：为一个 `int` 值，值大小对应两字符最小编辑距离

3. 人机接口

1. 文件输入输出调试:

```
dsl.exe SyntaxFile InputFile OutputFile
```

功能: 通过指定语法文件、输入文件和输出文件, 实现自动机的批量处理

参数:

SyntaxFile: 语法文件

InputFile: 输入文件

OutputFile: 输出文件

2. 交互式命令行调试:

```
dsl.exe SyntaxFile
```

功能: 用户输入语法文件名, 手动输入命令与自动机交互

参数:

SyntaxFile: 语法文件

2.3 测试桩设计说明

选择直接设置两个主要大类 `SyntaxParser` 与 `Automaton` 的子类 `SyntaxParserStub` 与

`AutomatonStub`

其中 `SyntaxParserStub` 模拟了 `SyntaxParser` 的核心功能, 例如返回固定的起始状态和状态数据

`AutomatonStub` 模拟了 `Automaton` 的行为, 例如输出固定的初始化信息并处理输入

通过这些测试桩, 可以在不依赖实际文件读取和复杂实现的情况下, 验证调用逻辑和数据流是否正确
具体的实现如下所示:

```
// SyntaxParser类测试桩
class SyntaxParserStub : public SyntaxParser {
public:
    SyntaxParserStub(const std::string& FileName) : SyntaxParser(FileName) {}

    bool CheckReady(void) {
        return true; // 固定返回语法树已就绪
    }

    void ShowSyntaxTree(void) {
        std::cout << "Syntax Tree Stub Output\n"; // 模拟输出
    }

    std::string AskBegin(void) const {
        return "StartState"; // 固定返回一个起始状态
    }

    std::pair<bool, State> AskState(const std::string& ask) {
        if (ask == "StartState") { // 模拟查询对应状态
            State testState;
            Action testAction;
            testAction.OutMessage.push_back("Output Message");
        }
    }
}
```



```

        testAction.NextState = "NextState";
        testState.ActionTable["TestInput"] = testAction;
        return {true, testState};
    }
    return {false, State()}; // 模拟查询失败
}

};

// Automaton类测试桩
class AutomatonStub : public Automaton {
public:
    AutomatonStub(const std::string& FileName, std::vector<std::string>& Buffer)
    : Automaton(FileName, Buffer) {
        Buffer.push_back("Automaton Initialized");
    }

    bool CheckReady(void) const {
        return true; // 固定返回自动机已就绪
    }

    void Input(const std::string& input, std::vector<std::string>& Buffer) {
        Buffer.push_back("Processed Input: " + input); // 模拟输入处理结果
    }

    void Exit(void) {
        std::cout << "Automaton Exited\n"; // 模拟退出自动机
    }
};

```

再设计 stub.cpp 程序调用以上测试桩内容，已验证程序逻辑设计是否合理：

```

#include "stubSyntax.h"

using namespace Syntax;

int main() {
    std::vector<std::string> buffer;

    // 创建 SyntaxParser 测试桩
    SyntaxParserStub parserStub("syntax1.txt");
    std::cout << "SyntaxParser CheckReady: " << parserStub.CheckReady() << "\n";
    parserStub.ShowSyntaxTree();
    std::cout << "Begin State: " << parserStub.AskBegin() << "\n";

    auto [found, state] = parserStub.AskState("StartState");
    if (found) {
        std::cout << "Found State with Input: TestInput\n";
        for (const auto& msg : state.ActionTable["TestInput"].OutMessage) {
            std::cout << "\tOutput Message: " << msg << "\n";
        }
    }
}

// 创建 Automaton 测试桩

```

```

AutomatonStub automatonStub("syntax1.txt", buffer);
std::cout << "Automaton CheckReady: " << automatonStub.CheckReady() << "\n";

automatonStub.Input("TestInput", buffer);
for (const auto& msg : buffer) {
    std::cout << msg << "\n";
}

automatonStub.Exit();
return 0;
}

```

运行以上测试桩测试程序，若诸输入输出均符合预期可以说明程序设计调用逻辑和数据流正确。

2.4 自动测试脚本设计说明

选择使用Windows批处理指令实现自动测试脚本

自动测试脚本设计思路如下：

1. 遍历 `input` 文件夹中的所有输入文件
2. 对每个输入文件运行 `dsl.exe`，将结果输出到 `output` 文件夹当中
3. 比较 `output` 文件夹和 `expected_output` 文件夹中的对应文件
4. 输出比较结果：是否一致

具体实现如下：

```

@echo off

setlocal enabledelayedexpansion

REM 设置文件夹路径
set SYNTAX_FILE=syntax\syntax1.txt
set INPUT_DIR=input
set OUTPUT_DIR=output
set EXPECTED_DIR=expected_output

REM 确保输出文件夹存在（清空旧结果）
if exist %OUTPUT_DIR% (
    echo 清空输出目录...
    del /q "%OUTPUT_DIR%\*"
) else (
    mkdir %OUTPUT_DIR%
)

echo =====
echo 开始自动测试...
echo =====

REM 遍历 input 文件夹中的所有文件
for %%F in (%INPUT_DIR%\*) do (
    set INPUT_FILE=%%F
    set FILE_NAME=%%~nF

```

```

set OUTPUT_FILE=%OUTPUT_DIR%\!FILE_NAME!.txt
set EXPECTED_FILE=%EXPECTED_DIR%\!FILE_NAME!.txt

echo 测试文件: !INPUT_FILE!

REM 执行 ds1.exe
ds1.exe %SYNTAX_FILE% !INPUT_FILE! !OUTPUT_FILE!

REM 检查程序是否成功输出文件
if not exist !OUTPUT_FILE! (
    echo [错误] 无法生成输出文件: !OUTPUT_FILE!
    goto :continue
)

REM 比较输出结果与预期输出
fc /b "!OUTPUT_FILE!" "!EXPECTED_FILE!" >nul
if !errorlevel! equ 0 (
    echo [成功] 测试通过: !FILE_NAME!
) else (
    echo [失败] 测试未通过: !FILE_NAME!
)

:continue
echo -----
)

echo =====
echo 测试完成!
echo =====

pause
endlocal

```

自动测试脚本解析：

变量设置

- `SYNTAX_FILE`：指定 `syntax` 文件的路径
- `INPUT_DIR`：输入文件夹路径
- `OUTPUT_DIR`：输出文件夹路径
- `EXPECTED_DIR`：预期输出文件夹路径

清空输出文件夹

- 使用 `del /q` 清空旧的输出文件

遍历输入文件

- `for %F in (%INPUT_DIR%\*)` 遍历 `input` 文件夹下的每个文件
- 通过 `%%~nF` 提取文件名，用于生成输出文件路径

执行 `ds1.exe`

- `ds1.exe %SYNTAX_FILE% !INPUT_FILE! !OUTPUT_FILE!`：运行程序，将结果输出到指定路径

比较结果

- 使用 `fc /b` 比较两个文件的内容：
- 检查 比较结果 `errorlevel`，若为 `0` 表示文件一致，否则文件不一致
- 输出匹配比较结果 `成功` 或 `失败`

3. 效果展示

使用以下命令编译程序：

```
g++ Syntax.cpp Automaton.cpp DSL.cpp -o ds1
```

3.1 自动测试脚本效果展示

3.1.1 测试语法说明

选择以下语法文件(`./Syntax/syntax1.txt`)作为基准（即实现手机运营商客服机器人基本问答功能，包含话费查询、线下营业厅查询、套餐查询、投诉等功能）：

```
STATE begin
    OUT "欢迎致电中国xx"
    OUT "这里是客服小祥，很高兴为您服务"
    OUT "请问有什么可以帮到您的"
    CASE "话费"
        NEXT queryFare
    CASE "营业厅"
        NEXT queryHall
    CASE "套餐"
        NEXT query
    CASE "投诉"
        NEXT complaint
    CASE "帮助"
        NEXT help
    CASE "没有"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    CASE "退出"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    CASE "再见"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    DEFAULT
        OUT "抱歉，我没有听清您的询问"
        OUT "请您复述您的需求"
        NEXT help
STATE help
    OUT "这边可以帮你查询以下业务，请输入您需要的服务"
    OUT "话费查询、营业厅查询、套餐查询与投诉"
    CASE "话费"
        NEXT queryFare
    CASE "营业厅"
```

```

        NEXT queryHall
CASE "套餐"
    NEXT query
CASE "投诉"
    NEXT complaint
DEFAULT
    OUT "抱歉，我没有听清您的询问"
    OUT "请您复述您的需求"
    NEXT help
STATE queryFare
    OUT "您的花费余额为xx元"
    NEXT other
STATE query
    OUT "当前有如下套餐"
    OUT "1.每月9元保号套餐"
    OUT "2.每月18元省内畅聊套餐"
    OUT "3.每月59元无限流量套餐"
    NEXT other
STATE queryHall
    OUT "登录移动客户端即可查询离您最近的营业厅位置"
    NEXT other
STATE complaint
    OUT "很抱歉给您带来了不好的体验"
    OUT "请您留下让您不满意的地方"
    DEFAULT
        OUT "感谢您的反馈，我们会尽快根据您的反馈进行改进"
        NEXT other
STATE other
    OUT "请问还有什么可以帮到您的吗"
    CASE "话费"
        NEXT queryFare
    CASE "营业厅"
        NEXT queryHall
    CASE "套餐"
        NEXT query
    CASE "投诉"
        NEXT complaint
    CASE "没有"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    CASE "退出"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    CASE "再见"
        OUT "感谢您的来电，祝您生活愉快"
        NEXT EXIT
    DEFAULT
        OUT "抱歉，我没有听清您的询问"
        OUT "请您复述您的需求"
        NEXT help

```

3.1.2 测试输入说明

测试文件如下：

测试内容1 (`./input/test1.txt`) ， 各输入均为已明确设置的期望输入内容：

进行一个话费的查
请问离我最近的营业厅在哪里
帮助
当前有哪些套餐
我要进行投那个诉
问营业厅在哪里还让我去查应用
没有了

测试内容2(`./input/test2.txt`), 存在未明确规定的输入内容，且已规定的检测内容中存在一定杂音干扰：

你有什么可以帮助我的
啊是第几啊送到家阿松大就哦啊圣诞节
话那个费那个查那个询
我喜欢你
不喜欢我？我要投诉你
有笨蛋
再见

测试内容3(`./input/test3.txt`), 各输入均存在大量杂音且未明确规定：

阿巴阿巴阿巴
歪比巴伯
话fffffffffffffffffff费
没有

3.1.3 测试预期输出说明

对以上各测试内容依据语法文件，进行人工设置期望输入，以进行匹配比较：

期望输出1(`./expected_output/test1.txt`), 期望均能直接按询问内容给出正确反馈

欢迎致电中国xx
这里是客服小祥，很高兴为您服务
请问有什么可以帮到您的
您的花费余额为xx元
请问还有什么可以帮到您的吗
登录移动客户端即可查询离您最近的营业厅位置
请问还有什么可以帮到您的吗
抱歉，我没有听清您的询问
请您复述您的需求
这边可以帮你查询以下业务，请输入您需要的服务
话费查询、营业厅查询、套餐查询与投诉
当前有如下套餐
1. 每月9元保号套餐
2. 每月18元省内畅聊套餐

3. 每月59元无限流量套餐

请问还有什么可以帮到您的吗

很抱歉给您带来了不好的体验

请您留下让您不满意的地方

感谢您的反馈，我们会尽快根据您的反馈进行改进

请问还有什么可以帮到您的吗

感谢您的来电，祝您生活愉快

期望输出2(./expected_output/test2.txt),期望能正确反馈可直接匹配上的询问内容，且对未规定的询问内容给出对应的 DEFAULT 反应

欢迎致电中国xx

这里是客服小祥，很高兴为您服务

请问有什么可以帮到您的

这边可以帮您查询以下业务，请输入您需要的服务

话费查询、营业厅查询、套餐查询与投诉

抱歉，我没有听清您的询问

请您复述您的需求

这边可以帮您查询以下业务，请输入您需要的服务

话费查询、营业厅查询、套餐查询与投诉

您的花费余额为xx元

请问还有什么可以帮到您的吗

抱歉，我没有听清您的询问

请您复述您的需求

这边可以帮您查询以下业务，请输入您需要的服务

话费查询、营业厅查询、套餐查询与投诉

很抱歉给您带来了不好的体验

请您留下让您不满意的地方

感谢您的反馈，我们会尽快根据您的反馈进行改进

请问还有什么可以帮到您的吗

感谢您的来电，祝您生活愉快

期望输出3(./expected_output/test3.txt),期望能对所有未规定的询问内容做出正确反应，且可以正确反应含杂音的询问

欢迎致电中国xx

这里是客服小祥，很高兴为您服务

请问有什么可以帮到您的

抱歉，我没有听清您的询问

请您复述您的需求

这边可以帮您查询以下业务，请输入您需要的服务

话费查询、营业厅查询、套餐查询与投诉

抱歉，我没有听清您的询问

请您复述您的需求

这边可以帮您查询以下业务，请输入您需要的服务

话费查询、营业厅查询、套餐查询与投诉

您的花费余额为xx元

请问还有什么可以帮到您的吗

感谢您的来电，祝您生活愉快

3.1.4 测试结果

运行自动测试脚本 `./auto_test.bat` 得到如下结果反馈：

```
清空输出目录...
=====
开始自动测试...
=====
测试文件：input\test1.txt
[成功] 测试通过：test1
-----
测试文件：input\test2.txt
[成功] 测试通过：test2
-----
测试文件：input\test3.txt
[成功] 测试通过：test3
-----
=====
测试完成！
=====
Press any key to continue . . .
```

各测试输入均能得到期望输出，表明程序实现正确可以符合预期。

接下来随机对某一期望输出进行修改，测试自动测试脚本的实现正确性：

选择修改期望输出1 `./expected_output/test1.txt`，将其内容任意删去一行，再次运行测试脚本，得到以下反馈：

```
清空输出目录...
=====
开始自动测试...
=====
测试文件：input\test1.txt
[失败] 测试未通过：test1
-----
测试文件：input\test2.txt
[成功] 测试通过：test2
-----
测试文件：input\test3.txt
[成功] 测试通过：test3
-----
=====
测试完成！
=====
Press any key to continue . . . |
```

可以看到修改过的测试文件1检测到输出结果匹配失败。反馈符合预期，表明自动测试脚本实现正确。

3.2 命令行交互效果展示

这里依旧挑用上文已给出的语法文件1 `./syntax/syntax1.txt` 作为基准使用以下命令进行运行测试：

```
dsl.exe syntax/syntax1.txt
```

输入各已明确定义的输入，均可得到期望的已定义输出：

```
D:\OneDrive\share\DSL>dsl.exe syntax/syntax1.txt
[21:17:06]欢迎致电中国xx
[21:17:06]这里是客服小祥，很高兴为您服务
[21:17:06]请问有什么可以帮到您的
[21:17:06]帮助查询
[21:17:22]这边可以帮你查询以下业务，请输入您需要的服务
[21:17:22]话费查询、营业厅查询、套餐查询与投诉
[21:17:22]话费查询
[21:17:25]您的花费余额为xx元
[21:17:25]请问还有什么可以帮到您的吗
[21:17:25]营业厅查询
[21:17:28]登录移动客户端即可查询离您最近的营业厅位置
[21:17:28]请问还有什么可以帮到您的吗
[21:17:28]套餐查询
[21:17:32]当前有如下套餐
[21:17:32]1.每月9元保号套餐
[21:17:32]2.每月18元省内畅聊套餐
[21:17:32]3.每月59元无限流量套餐
[21:17:32]请问还有什么可以帮到您的吗
```

输入含杂音的已定义输入信息，测试模糊匹配功能，亦可得到期望输出：

```
[21:17:32]话啊费撒询
[21:19:24]您的花费余额为xx元
[21:19:24]请问还有什么可以帮到您的吗
[21:19:24]额那什么营额业啊厅
[21:19:48]登录移动客户端即可查询离您最近的营业厅位置
[21:19:48]请问还有什么可以帮到您的吗
```

输入完全失配内容，也可以得到期望 `DEFAULT` 处理信息：

```
[21:19:48]asdasodaoisdjiaojd
[21:20:33]抱歉，我没有听清您的询问
[21:20:33]请您复述您的需求
[21:20:33]这边可以帮你查询以下业务，请输入您需要的服务
[21:20:33]话费查询、营业厅查询、套餐查询与投诉
[21:20:33]阿瑟东骄傲送到家睡觉哦埃及的
[21:20:36]抱歉，我没有听清您的询问
[21:20:36]请您复述您的需求
[21:20:36]这边可以帮你查询以下业务，请输入您需要的服务
[21:20:36]话费查询、营业厅查询、套餐查询与投诉
```

整体测试效果符合预期，程序设计符合需求。

3.3 测试桩效果展示

使用如下指令编译测试桩程序：

```
g++ AutomatonStub.cpp SyntaxStub.cpp stub.cpp -o stub
```

直接运行测试桩程序以进行测试，得到如下结果：

```
D:\OneDrive\share\DSL\stub>stub.exe
SyntaxParser CheckReady: 1
Syntax Tree Stub Output
Begin State: StartState
Found State with Input: TestInput
    Output Message: Output Message
Automaton CheckReady: 1
Automaton Initialized
Processed Input: TestInput
Automaton Exited
```

测试桩测试结果亦符合预期。

4. 总结

本次实验使用C++语言设计并实现了一个简单的脚本语言以及其对应的解释器并在此基础上实现完成了DSL。

本次实验实现起来相较于以往的大作业顺利了许多，除去初期的设计部分，基本上很顺畅的想如何做便成功的实现了对应的功能，偶尔存在的一些小问题也很快的通过调试修改成功。这也许是我代码实现能力逐渐进步的体现吧。

当然，本次实验也还是令我收益良多，许许多多以前未曾关注在意的细节都在这里得到了完善，如：良好的命名风格、良好的注释习惯。先前我所写的代码更多的是竞赛性的一次性代码，故而命名风格与注释常常是被忽略的。本次实验让我建立起了良好的工程代码习惯。

同时，我也是首次使用批处理命令进行自动化的测试文件，这也是我所新学习到的内容。

以及测试桩这一重要概念的学习以及实践上手操作设计并实现的过程也让我学习到了诸多工程测试理念与方法。

总而言之，本次实验实是令我受益匪浅。

5. 文件说明

```
-expected_output // 自动测试期望输出文件夹
-input // 自动测试输入文件夹
-output // 自动测试输出文件夹
-stub // 测试桩文件夹
-syntax // 语法文件文件夹
auto_test.bat // 自动测试脚本
Automaton.cpp // 程序源代码自动机实现部分
DSL.cpp // 程序源代码，main函数实现部分
dsl.exe // 可执行文件
Syntax.cpp // 程序源代码，语法树实现部分
Syntax.h // 程序源代码头文件
实验报告.pdf // 实验报告
```